

Projeto Final II — Contexto Completo do Projeto

Documento de contexto para uso com ferramentas e agentes de IA durante o desenvolvimento do MVP.

1. Visão geral

O projeto é o desenvolvimento de uma plataforma web para centralização e análise de informações relacionadas à gestão de projetos em empresas de consultoria de tecnologia. O projeto nasceu de um problema observado no contexto profissional: informações importantes de um mesmo projeto podem ficar distribuídas entre diferentes ferramentas, documentos e fontes, dificultando uma visão consolidada para gestores.

A proposta é criar uma plataforma interna que reúna informações essenciais dos projetos em um único ambiente e transforme esses dados em indicadores e visualizações que apoiem o acompanhamento, a identificação de riscos e a tomada de decisão.

O projeto faz parte do Projeto Final II do curso de Análise e Desenvolvimento de Sistemas da Unisinos. O Projeto Final I foi dedicado à definição do problema, objetivos, fundamentação teórica, trabalhos relacionados e planejamento metodológico. O Projeto Final II coloca a proposta em prática por meio do desenvolvimento e avaliação de um MVP.

2. Problema

Em empresas de consultoria tecnológica, diferentes projetos podem envolver clientes, equipes, responsáveis, orçamento, horas trabalhadas, prazos e objetivos distintos. Quando essas informações estão dispersas, o gestor precisa consultar diferentes fontes para obter uma visão geral do projeto.

Essa fragmentação dificulta o acompanhamento do desempenho, a identificação de situações de risco e a tomada de decisões baseada em informações consolidadas.

O projeto não pretende substituir todas as ferramentas existentes de gestão. O foco do MVP é criar uma camada centralizada de informações gerenciais, com visualização e análise dos dados mais relevantes.

3. Questão de pesquisa

Como a centralização e análise de informações de projetos em uma plataforma única podem contribuir para melhorar a gestão de projetos em empresas de consultoria de tecnologia?

4. Objetivo geral

Desenvolver uma plataforma para centralização e análise de informações de projetos, com o objetivo de apoiar a gestão de projetos e melhorar a eficiência operacional em empresas de consultoria de tecnologia.

5. Objetivos específicos

- Identificar as principais informações e indicadores utilizados na gestão de projetos em empresas de tecnologia.
- Analisar os desafios relacionados à dispersão de informações em diferentes ferramentas utilizadas na gestão de projetos.
- Projetar a arquitetura de uma plataforma capaz de centralizar dados relacionados à gestão de projetos.
- Desenvolver funcionalidades que permitam a visualização e análise de indicadores por meio de dashboards e relatórios.
- Avaliar como a centralização dessas informações pode contribuir para melhorar o acompanhamento e a tomada de decisões na gestão de projetos.

6. Escopo do MVP

O MVP será deliberadamente simples e funcional. A prioridade é comprovar a proposta central do projeto sem transformar o TCC em um sistema corporativo de grande escala.

- Cadastro de clientes.
- Cadastro de equipes.
- Cadastro de usuários e perfis.
- Cadastro, consulta e atualização de projetos.
- Visão detalhada de um projeto.
- Dashboard gerencial com indicadores.
- Indicadores relacionados a orçamento, horas, prazo, status, clientes e projetos em risco.
- Usuário logado simulado, sem implementação de autenticação real.
- Dados fictícios inseridos manualmente pelos usuários para representar o funcionamento da plataforma.
- Não haverá NPS no MVP.
- Não haverá timesheet individual; será armazenado somente o total de horas realizadas no projeto.
- Não haverá controle individual dos membros de uma equipe.
- IA generativa, agentes e funcionalidades avançadas de recomendação não fazem parte do núcleo obrigatório do MVP.

7. Usuários

Os perfis previstos para utilização da plataforma são:

- Gerente.
- Coordenador.
- Gestor de Projeto.

No MVP, a autenticação não será implementada. O sistema poderá utilizar um usuário simulado para representar o contexto de acesso e permitir o desenvolvimento das funcionalidades sem adicionar complexidade desnecessária.

8. Dados principais de um projeto

Cada projeto terá um único cliente, um gestor responsável e uma equipe. Uma equipe poderá participar de vários projetos.

- Nome do projeto.
- Cliente.
- Objetivo do projeto.
- Gestor responsável.
- Equipe.
- Data de início.
- Prazo final.
- Orçamento previsto.
- Orçamento consumido.
- Total de horas realizadas.
- Status.
- Observações.

9. Status

- PLANEJAMENTO
- EM_ANDAMENTO
- EM_RISCO
- CONCLUÍDO
- CANCELADO

Os status serão armazenados como texto e validados no backend. A escolha evita a necessidade de alterar o tipo do banco durante o desenvolvimento caso o escopo do MVP seja refinado.

10. Indicadores e dashboard

O dashboard será integrado ao frontend da aplicação. Não será tratado como uma aplicação separada. Os gráficos e indicadores serão componentes da interface web, consumindo os dados disponibilizados pelo backend.

- Quantidade total de projetos.

- Projetos por status.
- Projetos por cliente.
- Projetos em risco.
- Projetos atrasados.
- Orçamento total.
- Orçamento consumido.
- Percentual de consumo do orçamento por projeto.
- Projetos com orçamento excedido.
- Total de horas trabalhadas.
- Visão geral de um projeto específico.

Os indicadores derivados serão calculados pela aplicação e não serão armazenados como campos independentes no banco. Por exemplo, o percentual de orçamento consumido será calculado a partir de $\text{budget_spent} / \text{budget} \times 100$.

11. Arquitetura da solução

A solução será construída seguindo uma arquitetura web cliente-servidor separada em frontend e backend. O frontend será responsável pela interface e visualização dos dados; o backend concentrará as regras de negócio e disponibilizará uma API REST; o PostgreSQL será responsável pela persistência.

Fluxo conceitual:

Frontend React → API REST Node.js → Camada de negócio → Prisma ORM → PostgreSQL

A separação facilita a manutenção e permite que frontend e backend sejam desenvolvidos e testados de forma independente.

12. Stack tecnológica

12.1 Frontend

- React.
- TypeScript.
- Biblioteca de componentes/UI a ser escolhida de acordo com simplicidade e produtividade.
- Biblioteca de gráficos para os dashboards, escolhida durante a implementação.
- Consumo da API REST do backend.

12.2 Backend

- Node.js.
- TypeScript.
- API REST.
- Prisma ORM.
- Validação de dados e regras de negócio no backend.

12.3 Banco de dados

- PostgreSQL.
- UUID como identificador das entidades.
- Relacionamentos por chaves estrangeiras.
- Prisma Migrations para versionamento da estrutura do banco.

12.4 Ferramentas de desenvolvimento

- Git para versionamento.
- GitHub ou GitLab para hospedagem dos repositórios.
- Visual Studio Code ou ambiente equivalente.
- Ferramentas de IA para apoio ao desenvolvimento, revisão, geração de código, testes e documentação.

A escolha tecnológica prioriza ferramentas conhecidas, simples de manter e adequadas ao desenvolvimento acelerado com apoio de IA.

13. Estrutura dos repositórios

O projeto será dividido inicialmente em dois repositórios:

- Frontend: aplicação React + TypeScript.
- Backend: API Node.js + TypeScript.

O banco PostgreSQL será utilizado pelo backend. O schema do banco será versionado no repositório do backend por meio das migrations do Prisma.

14. Modelagem de dados

O banco possui quatro tabelas principais:

- users
- clients
- teams
- projects

A tabela projects é a entidade central. Ela possui as chaves estrangeiras client_id, manager_id e team_id.

15. Tabela users

- id — UUID — PK
- name — VARCHAR(100) — NOT NULL
- email — VARCHAR(150) — NOT NULL, UNIQUE
- role — VARCHAR(30) — NOT NULL
- created_at — TIMESTAMP — NOT NULL

Roles previstos: GERENTE, COORDENADOR e GESTOR_PROJETO.

16. Tabela clients

- id — UUID — PK
- name — VARCHAR(150) — NOT NULL
- created_at — TIMESTAMP — NOT NULL
- updated_at — TIMESTAMP — NOT NULL

17. Tabela teams

- id — UUID — PK
- name — VARCHAR(100) — NOT NULL
- created_at — TIMESTAMP — NOT NULL

18. Tabela projects

- id — UUID — PK
- name — VARCHAR(150) — NOT NULL
- client_id — UUID — FK para clients.id
- objective — TEXT — NOT NULL
- manager_id — UUID — FK para users.id
- team_id — UUID — FK para teams.id
- start_date — DATE — NOT NULL
- deadline — DATE — NOT NULL
- budget — NUMERIC(15,2) — NOT NULL
- budget_spent — NUMERIC(15,2) — NOT NULL, DEFAULT 0
- hours_worked — NUMERIC(10,2) — NOT NULL, DEFAULT 0
- status — VARCHAR(30) — NOT NULL
- observations — TEXT — NULL
- created_at — TIMESTAMP — NOT NULL
- updated_at — TIMESTAMP — NOT NULL

19. Regras de negócio principais

- O orçamento deve ser maior ou igual a zero.
- O orçamento consumido deve ser maior ou igual a zero.
- O sistema não deve impedir que o orçamento consumido ultrapasse o orçamento previsto, pois essa situação é relevante para análise.
- As horas realizadas devem ser maiores ou iguais a zero.
- A data de término prevista deve ser igual ou posterior à data de início.
- Todo projeto deve possuir cliente, gestor, equipe, objetivo, prazo, orçamento e status.
- Um projeto possui somente um cliente.

- Um projeto possui somente um gestor responsável.
- Um projeto possui uma equipe responsável.
- Uma equipe pode estar associada a vários projetos.

20. Indicadores derivados

- Consumo do orçamento = $\text{budget_spent} / \text{budget} \times 100$.
- Projeto atrasado = data atual posterior ao deadline e projeto não concluído/cancelado.
- Orçamento excedido = $\text{budget_spent} > \text{budget}$.
- Projetos em risco = projetos com status EM_RISCO e/ou condições de atenção apresentadas pelos indicadores.

Os indicadores serão calculados no backend ou na camada de apresentação conforme a necessidade. Não devem ser duplicados como campos persistidos sem justificativa.

21. Requisitos funcionais principais

- RF01 — Cadastrar usuário.
- RF02 — Cadastrar cliente.
- RF03 — Cadastrar projeto.
- RF04 — Consultar projetos.
- RF05 — Consultar detalhes de um projeto.
- RF06 — Atualizar projeto.
- RF07 — Exibir indicadores dos projetos.
- RF08 — Exibir dashboard gerencial.
- RF09 — Identificar projetos em situação de risco.

22. Requisitos não funcionais principais

- RNF01 — Usabilidade: interface clara e organizada.
- RNF02 — Desempenho: consultas e indicadores devem apresentar tempo de resposta adequado para o MVP.
- RNF03 — Segurança: estrutura preparada para controle de acesso, embora autenticação real não seja implementada no MVP.
- RNF04 — Manutenibilidade: código organizado e arquitetura que permita evolução.
- RNF05 — Integridade dos dados: relacionamentos e validações devem preservar a consistência das informações.

23. Telas previstas

- Dashboard principal.
- Lista de projetos.
- Cadastro de projeto.

- Detalhes do projeto.
- Cadastros auxiliares de clientes, equipes e usuários, conforme necessidade da implementação.

A tela de detalhes do projeto deverá apresentar informações como nome, cliente, objetivo, gestor, equipe, orçamento, percentual consumido, horas realizadas, prazo, status e observações, além dos indicadores e gráficos considerados relevantes.

24. Estratégia de desenvolvimento

O desenvolvimento seguirá uma abordagem incremental. Primeiro será construída a base do backend e do banco de dados; depois serão implementados os recursos de cadastro e consulta; em seguida será desenvolvido o frontend; por fim, os dashboards, indicadores e mecanismos de avaliação.

- Configurar repositórios e ambiente.
- Configurar PostgreSQL e Prisma.
- Criar schema e migrations.
- Implementar entidades e regras no backend.
- Implementar endpoints REST.
- Implementar frontend base.
- Implementar telas de CRUD.
- Integrar frontend e backend.
- Implementar dashboard e indicadores.
- Gerar dados fictícios para demonstração.
- Testar e corrigir o sistema.
- Preparar a validação com profissionais da empresa.
- Documentar resultados para o artigo.

25. Validação do projeto

A solução deverá ser avaliada com usuários que possuam experiência em gestão de projetos, como gerente de operações, coordenadores e gestores de projeto. A avaliação deverá verificar principalmente se a plataforma facilita a visualização das informações, melhora a organização dos dados e auxilia o acompanhamento e a tomada de decisão.

A estratégia de avaliação será detalhada no Capítulo 4 e aplicada para produzir os resultados e discussões do Capítulo 5.

26. Uso de Inteligência Artificial no desenvolvimento

Ferramentas de IA poderão ser utilizadas como apoio ao desenvolvimento, principalmente para geração de código, criação de testes, revisão, documentação, refatoração e investigação de problemas. A IA será utilizada como ferramenta de produtividade e não como substituta da definição dos requisitos, decisões de arquitetura ou validação do projeto.

A possibilidade de utilizar IA posteriormente para análise avançada dos dados, geração de insights ou recomendações é uma oportunidade de evolução, mas não constitui requisito obrigatório do MVP. O núcleo do trabalho continua sendo a centralização e análise das informações de gestão de projetos.

27. Princípios para as IAs de desenvolvimento

- Não adicionar funcionalidades fora do escopo sem solicitação.
- Priorizar soluções simples e funcionais.
- Evitar overengineering.
- Manter TypeScript como linguagem principal no frontend e backend.
- Manter a separação entre frontend, backend e banco.
- Não criar tabelas ou campos sem necessidade funcional.
- Não implementar autenticação real no MVP.
- Não introduzir IA generativa como requisito obrigatório do sistema.
- Antes de alterar a arquitetura, explicar o motivo e o impacto.
- Priorizar código legível, testável e fácil de manter.
- Gerar testes para regras de negócio importantes.
- Manter migrations versionadas.
- Não inserir credenciais ou segredos diretamente no código.
- Atualizar a documentação quando decisões estruturais forem alteradas.

28. O que não faz parte do MVP

- Autenticação e autorização reais.
- Controle individual de membros das equipes.
- Timesheet detalhado por colaborador.
- NPS.
- Integrações com sistemas corporativos reais.
- Importação automática de dados.
- Agentes autônomos de IA.
- Recomendações comerciais automatizadas.
- Arquitetura distribuída ou microsserviços.
- Infraestrutura de produção corporativa.

Esses itens podem aparecer como possibilidades de trabalhos futuros, desde que façam sentido após a avaliação do MVP.

29. Diretriz principal do projeto

A plataforma deve demonstrar, de forma prática e mensurável, que centralizar informações essenciais dos projetos em um único ambiente pode facilitar o acompanhamento gerencial e apoiar a análise e a tomada de decisões. O projeto deve priorizar uma solução simples,

funcional e validável, evitando transformar o MVP em uma plataforma corporativa excessivamente complexa.